

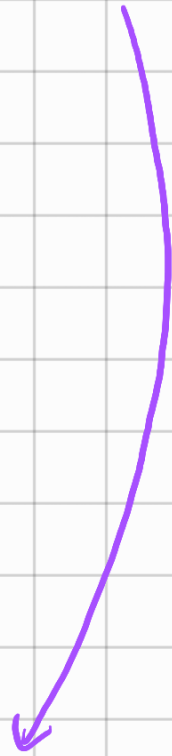
Search algorithms

Sorted arrays

- Linear Search
- Binary Search
- jump search
- Interpolation Search
- exponential Search

Unsorted Arrays

Linear Search



5	1	2	3	6	4
---	---	---	---	---	---

$N=6$

$X=5 \rightarrow 1$
 $X=8 \rightarrow X$

Linear search

Start

$N, A[N], x$

$I = 1$

$I \leq N$ And $A[I] \neq x$

$I = I + 1$

$I \leq N$

Not found

I

→ eger bu koda
girsene demek
ki while
içindeki 2.koşul
doğrulmuş ki
yani x 'i

Stop

bu Emu's demektir

eger bu kola pırsese
demek ki x'i bulamamış
demektir

<u>I</u>	<u>A[i]</u>	<u>x</u>	<u>N</u>
----------	-------------	----------	----------

1	5	3	6
---	---	---	---

2	1	3	6
---	---	---	---

3	2	3	6
---	---	---	---

4	3	3	
---	---	---	--

1	5	4	6
---	---	---	---

2	1	⋮	⋮
---	---	---	---

3	2	⋮	⋮
---	---	---	---

4	3	⋮	⋮
---	---	---	---

5	6	⋮	⋮
---	---	---	---

6	4	4	⋮
---	---	---	---

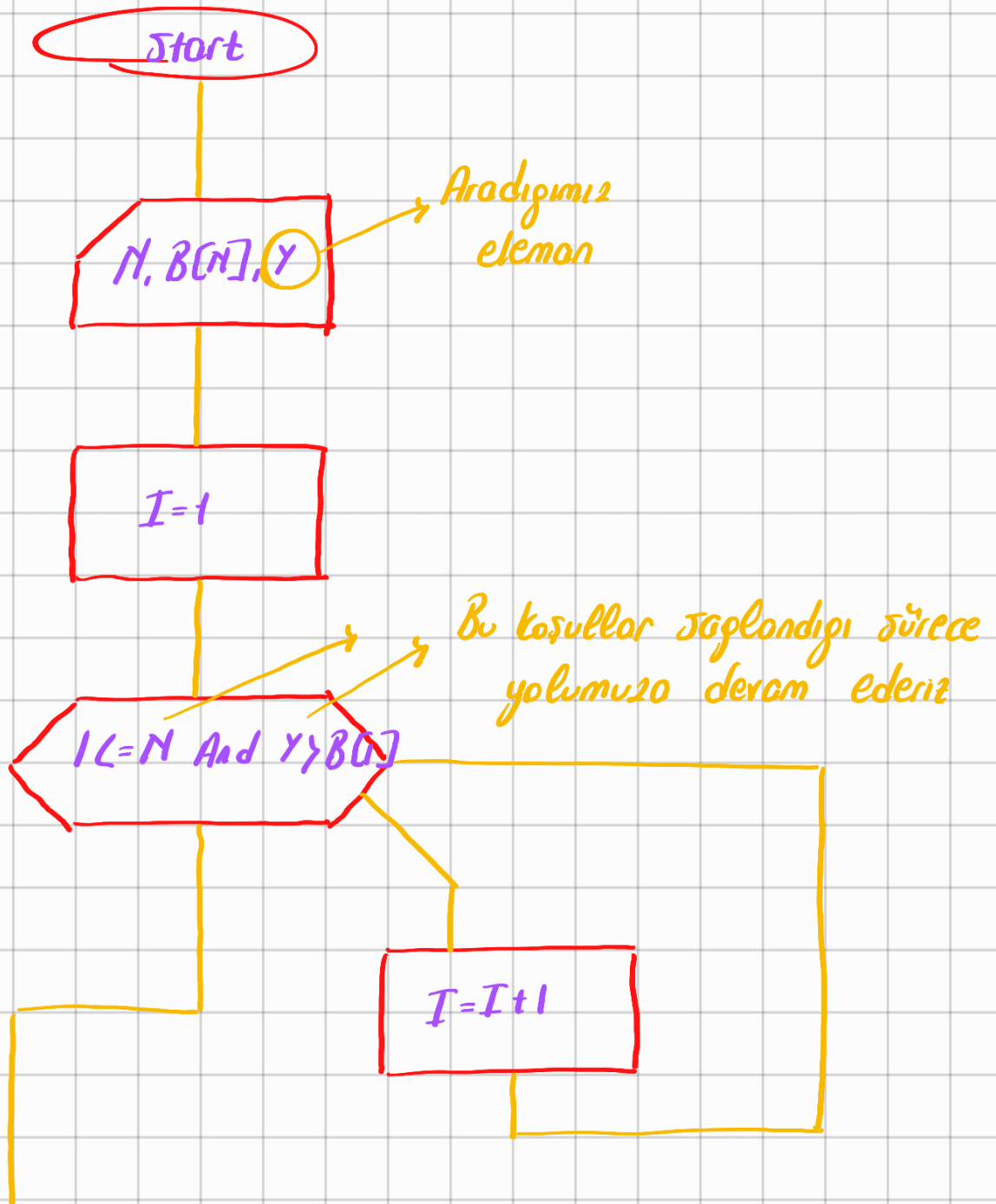
7	7
---	---

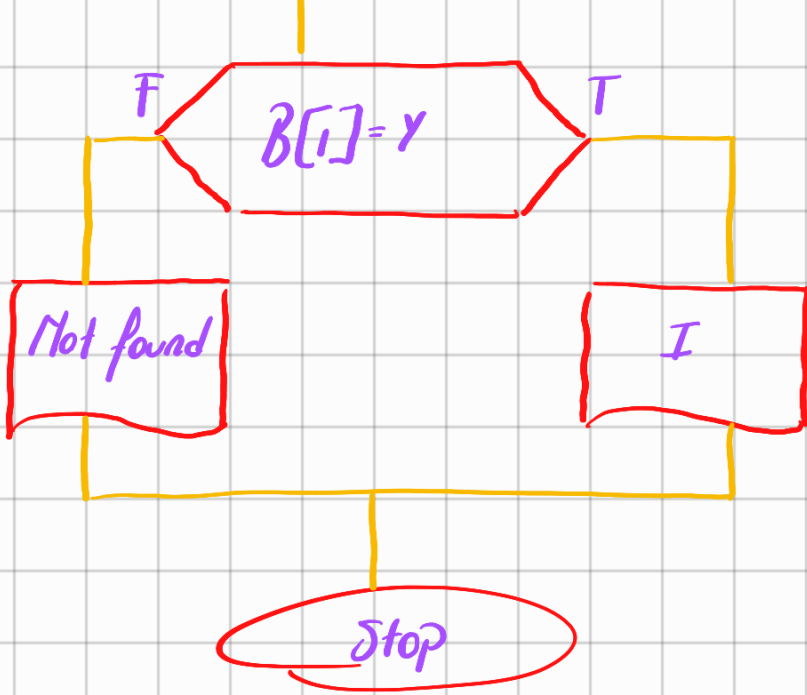
Not found durumu
çünkü dizi sonuna
ulaşmış

5	1	2	3	6	4
---	---	---	---	---	---

Verilen bir elemanın varsa yerini yoksa bulunmadığı bilgisini ekrana yazdıran algoritmanın okuş diagramını çiziniz (sıralı bir dizi)

1	2	3	4	5	6	7
1	3	5	8	12	25	32





1 ✓ } Bu ikisi
32 ✓ } için çalışır

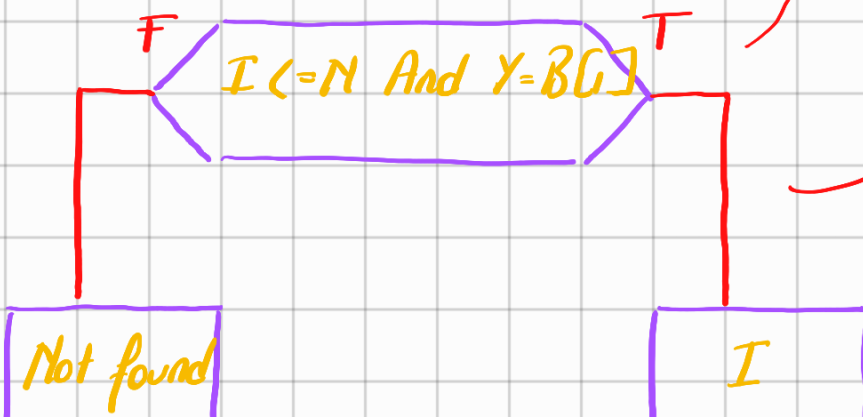
35 x

→ ama bu ifade için algoritma çalışmaz

Bu algoritma elemanın dizideki elemanlardan büyük olduğu durum için çalışmaz

$B[8] = Y$

durumu yapıyor çünkü bizim dizimiz 7 elemanlı



→ Böyle extra bir koşul ekleyerek çalıştırabiliriz

Stop

Peki bu if'in içine fazladan bir şart koymadan algoritmamızı nasıl yaparız

Start

$N, B[N], y$

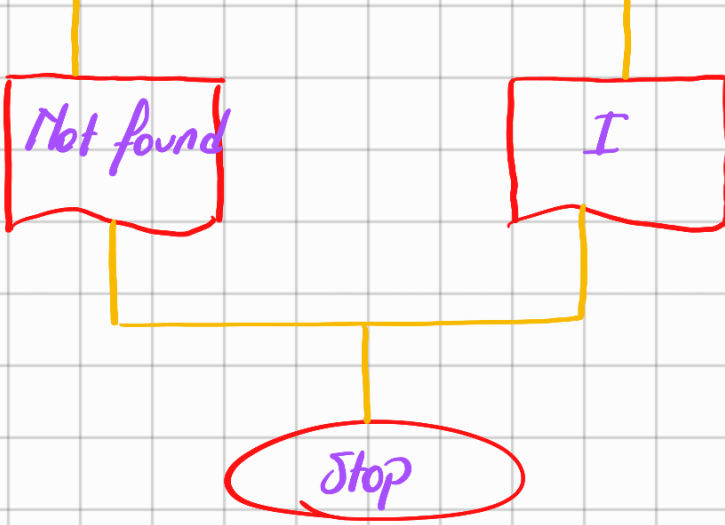
$I = 1$

$I \neq N$ And $y > B[I]$

$I = I + 1$

$y = B[I]$

İon elemanı while'da
değil if'de kontrol



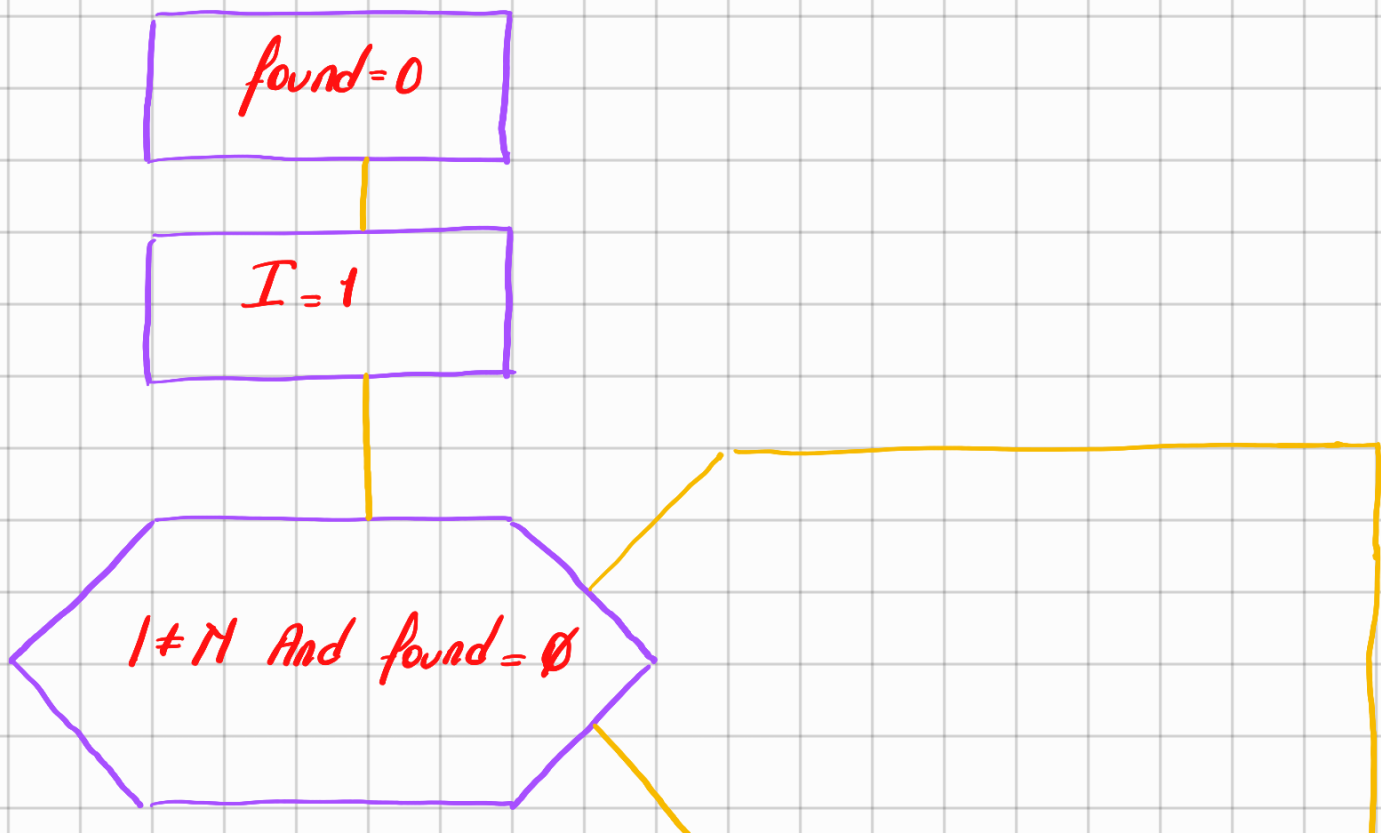
aynı 4 ve 5. eleman
ediyoruz Bu sağda
dizinin dışına
Çıkıyoruz

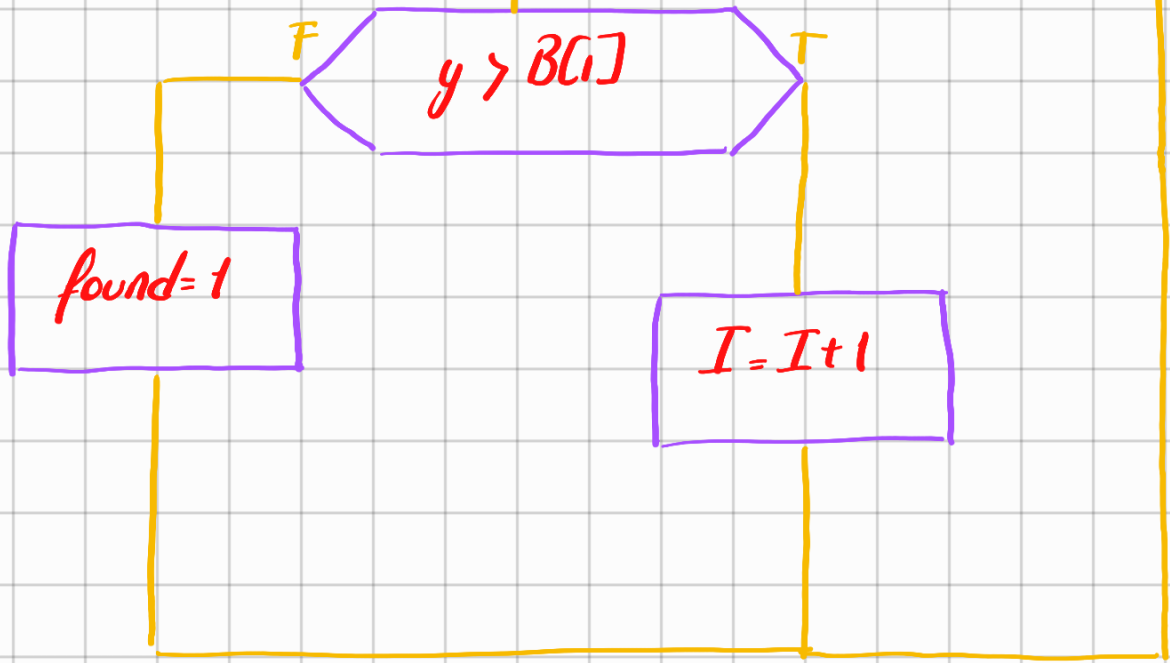
1 v v
4 x v
32 v v
35 v v

fazladan sort eklemeden
de istediğimiz algoritma-
yı çözmüş olduk

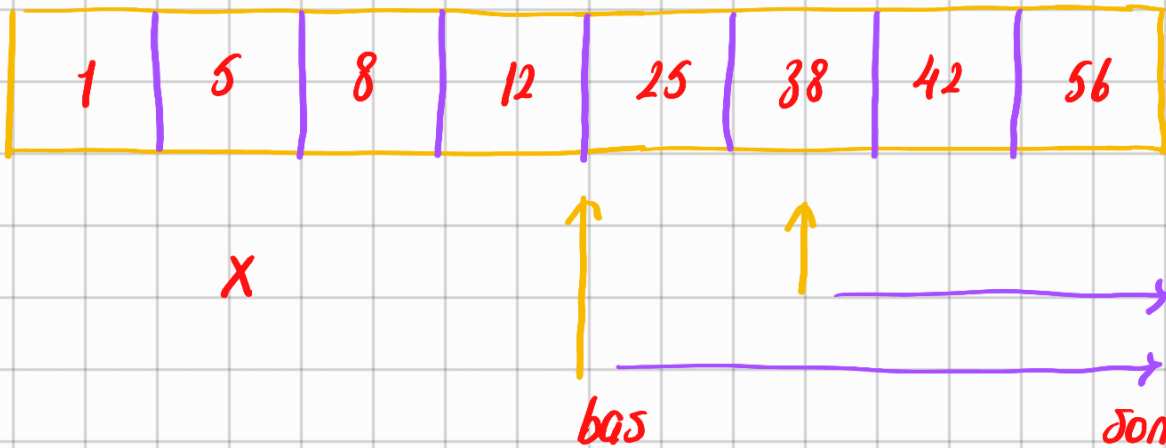
Bu algoritma farklı bir formatta
da yapılabilirdi

Algoritmanın while durumunu böyle de
yapabilirdin





Binary Search algoritması (elementler burada
 $(\log_2 N)$ sıralı olmak zorunda)



Bu dizinin ortasından başlanıp arıyorum aradığım sayı 42
 0. aşamada dizinin 1. yarısını ele ve 2. yarısını bırakarak ara

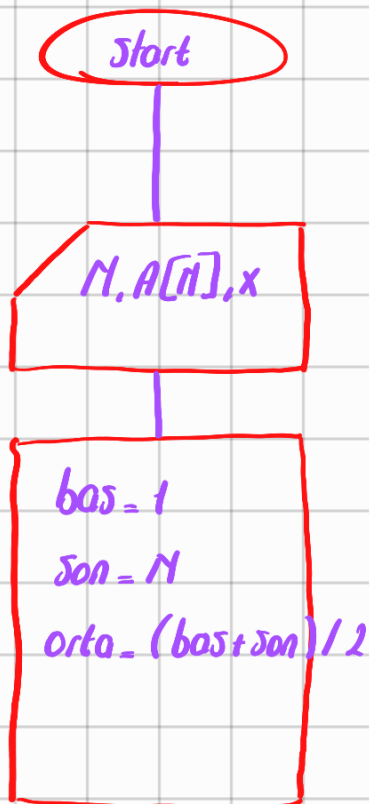
2. zaman aralımı 1. gösterimi alır ve 2. gösterimi bulurken sonra 38'e gelir ve ona bakarsın baktın bu sayıdan da büyük o zaman 38'inde sağına bakarsın

Linear search karmaşıklığı
 N

Binary search karmaşıklığı
 $\log_2 N$

yani Linear search'de 100 adımda yapacağın işi Binary Search'de 7 adımda yapabilirsin

Algoritması



1

1

2

$bas \leq son$ And $x \neq A[orta]$

F

$x > A[orta]$

T

$son = orta - 1$

$bas = orta + 1$

$orta = (bas + son) / 2$

2

F

$bas \leq son$

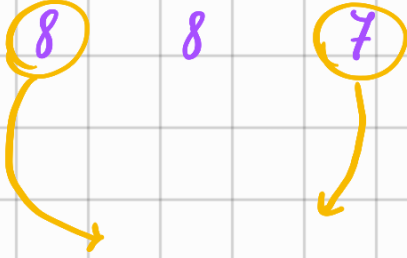
T

Not found

Orta

Stop

Baş	orta	son	N	x	A[orta]
1	4	8	8	42	12
5	6	8			38
7	7	8		42	42
8	8	8		43	56



Burada $baş \leq son$
koluna gider ve
kısul sağlanmadığı
için false koluna
gider ve not found
diye çıkar

time Complexity (worst case)

linear search

Binary search

found	$O(N)$	$O(\log_2 N)$
Not found	$O(N+1)$	$O(\log_2 N+1)$
found	$N/100$	7
	1000	10

Sıralı bir diziye while kullanmak lazım

Sırasız bir diziye for kullanmak daha mantıklı

Direct Address table

hash table

Sırasız bir diziye binary search uygulamak için sıralama maliyeti

↳ $O(N \log N)$

↳ Sonra buna binary search uygulamak $O(\log N)$

yani

$O(N \log N) + O(\log N)$

↳ Büyük olan olur
 $O(N \log N)$

Sırası2 bir diziye binary
search uygulamanın
maliyeti budur

İşaretsiz tam sayı sisteminde

1 ve 0'lerden oluşan 2'li
sayı sisteminde sayıla-
rın binary karşılık-
ları var

1	0	1	0	0	1	1	0
1	0	1	0	0	1		
0	1	1	1	1	1	1	1
1	0	0	0	1	0	0	0
0	0	0	0	0	0	0	1

Bunların onluk sayı siste-
mindeki karşılıklarını
bir diziye yazdıran
algoritmanın akış
diagramını çiziniz

		127	-120		1
--	--	-----	------	--	---

10001000
01110111
1

+

01111000

64 32 16 8

-120

en soldaki bite bakarız 1 ise 256'dan çıkarır
ve ekşi işareti koyarız 0 ise direk
hesaplarız

Bunun kodunu yaz

Jump Search

1 5 8 12 96 172 251 255 3000
↑ ↑ ↑

Belirli adımlarla atlayarak elemanı arıyorsunuz

Sonra atıyorum 96 çıktı 12 ve 251 arasında bu iki yer
arasında linear search yaparsın

Atlama aralıklarını dizinin uzunluğuna göre belirlerdin

$\sqrt{n}$ bizim atlama sayımızı temsil ederse en optimum
düzeyde elemana ulaşma şansına sahip oluyoruz

Bu algoritmanın karmaşıklığı
 $O(\sqrt{n})$